

École Nationale des Sciences Appliquées d'AI Hoceima
Université Abdelmalek Essaadi

Compte Rendu de Travaux Pratiques

Module : Java EE (JEE)

Application de Gestion des Produits

MVC2 — Annotations JPA vs Fichier hbm.xml

Deux approches de mapping Hibernate · web.xml vs @WebServlet · POJO pur vs @Entity

Réalisé par	Nabila Es-Saidy
Encadrant	M. Cherradi
Année	2024 – 2025

ENSA AI Hoceima — Filière Génie Informatique

Introduction

Ce compte rendu compare deux variantes d'une même application web Java EE de gestion de produits, toutes deux basées sur le patron MVC2, mais utilisant deux approches différentes pour le mapping Hibernate :

- MVC2-Annotations : le mapping objet-relationnel est décrit directement dans le code Java via les annotations JPA (`@Entity`, `@Table`, `@Id`, `@Column...`). Les Servlets et filtres sont aussi configurés par annotations (`@WebServlet`, `@WebFilter`).
- MVC2-hbm.xml : le mapping est externalisé dans un fichier XML dédié (`Produit.hbm.xml`), séparant complètement la configuration de persistance du code Java. Les Servlets et filtres reviennent à une déclaration dans `web.xml`.

Les deux projets ont exactement les mêmes fonctionnalités, la même architecture en couches (DAO / Service / Servlet / JSP), et le même comportement utilisateur. La différence est purement technique et porte sur la manière dont Hibernate connaît le mapping entre la classe `Produit` et la table MySQL.

1. Présentation générale des deux variantes

1.1 Fonctionnalités (identiques dans les deux variantes)

Fonctionnalité	Description	Rôle requis
Authentification	Connexion login/mot de passe avec session HTTP	Tous
Liste des produits	Affichage depuis MySQL via Hibernate	Tous
Recherche par ID	Filtrage de la liste par identifiant	Tous
Ajout de produit	INSERT via Hibernate	ADMIN
Modification	UPDATE via Hibernate	ADMIN
Suppression	DELETE via Hibernate + confirmation JS	ADMIN
Erreur 403 / 404 / 500	Gestion centralisée des erreurs HTTP	Tous
Déconnexion	Invalidation de session	Tous

1.2 Structures comparées des deux projets

MVC2-Annotations — structure	MVC2-hbm.xml — structure
<pre> src/main/ ├── java/ma/ensa/ │ ├── models/ │ │ └── Produit.java ← @Entity JPA │ ├── dao/ │ │ ├── HibernateUtil.java │ │ └── ProduitDAOImpl.java │ ├── filters/ │ │ ├── AuthFilter.java ← │ │ └── RoleFilter.java ← │ ├── @WebFilter │ │ └── web/ │ │ ├── ProduitServlet.java ← │ │ ├── LoginServlet.java ← │ │ └── LogoutServlet.java ← │ ├── @WebServlet │ │ └── resources/ │ │ ├── hibernate.cfg.xml │ │ └── <mapping class="..."/> │ webapp/WEB-INF/web.xml └── Minimal (erreurs HTTP seulement) </pre>	<pre> src/main/ ├── java/ma/ensa/ │ ├── models/ │ │ └── Produit.java ← POJO pur │ ├── dao/ │ │ ├── HibernateUtil.java │ │ └── ProduitDAOImpl.java │ ├── filters/ │ │ ├── AuthFilter.java ← sans │ │ └── RoleFilter.java ← sans │ ├── annotation │ │ └── web/ │ │ ├── ProduitServlet.java ← sans │ │ ├── LoginServlet.java ← sans │ │ └── LogoutServlet.java ← sans │ ├── annotation │ │ └── resources/ │ │ ├── hibernate.cfg.xml │ │ ├── <mapping resource="..."/> │ │ └── ma/ensa/models/ │ │ └── Produit.hbm.xml ← NOUVEAU │ webapp/WEB-INF/web.xml └── Complet (servlets+filtres+erreurs) </pre>

2. Tableau comparatif global

Aspect	MVC2-Annotations	MVC2-hbm.xml
Mapping Hibernate entité	Annotations JPA dans Produit.java (@Entity, @Table, @Id...)	Fichier XML externe Produit.hbm.xml
Référence dans hibernate.cfg.xml	<mapping class="ma.ensa.models.Produit"/>	<mapping resource="ma/ensa/models/Produit.hbm.xml"/>
Classe Produit.java	POJO + annotations JPA (couplée à Hibernate)	POJO pur, aucune annotation, aucun import Hibernate
Configuration Servlets	@WebServlet sur chaque classe	Déclarées dans web.xml (<servlet> + <servlet-mapping>)
Configuration Filtres	@WebFilter sur chaque classe	Déclarés dans web.xml (<filter> + <filter-mapping>)
web.xml	Minimal — erreurs HTTP uniquement	Complet — servlets, filtres et erreurs
Fichiers de configuration	1 fichier (hibernate.cfg.xml)	2 fichiers (hibernate.cfg.xml + Produit.hbm.xml)
Couplage modèle/persistance	Fort (annotations dans le POJO)	Faible (XML séparé du POJO)
Lisibilité du mapping	Visible directement dans le code Java	Dans un fichier XML dédié, séparé
Modification du schéma BDD	Modifier la classe Java (annotations)	Modifier uniquement le fichier XML
Imports javax.persistence	Nécessaires dans Produit.java	Absents — POJO totalement propre
Code ProduitDAOImpl	Identique dans les deux variantes	Identique dans les deux variantes
Services / Couche métier	Identiques dans les deux variantes	Identiques dans les deux variantes
Vues JSP	Identiques dans les deux variantes	Identiques dans les deux variantes

3. Différence principale : le mapping Hibernate

C'est la différence fondamentale entre les deux variantes. Elle affecte trois artefacts : la classe `Produit.java`, le fichier `hibernate.cfg.xml`, et l'existence ou non d'un fichier `Produit.hbm.xml`.

3.1 La classe `Produit.java`

MVC2-Annotations — <code>Produit.java</code>	MVC2-hbm.xml — <code>Produit.java</code>
<pre>package ma.ensa.models; import javax.persistence.*; // Annotations JPA sur la classe @Entity @Table(name = "produits") public class Produit { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) @Column(name = "id_produit") private Long idProduit; @Column(name = "nom", nullable = false, length = 150) private String nom; @Column(name = "description", length = 500) private String description; @Column(name = "prix", nullable = false) private Double prix; public Produit() {} // Constructeur, getters, // setters... }</pre>	<pre>package ma.ensa.models; // AUCUN import javax.persistence // AUCUNE annotation // POJO totalement pur public class Produit { private Long idProduit; private String nom; private String description; private Double prix; public Produit() {} public Produit(String nom, String description, Double prix) { this.nom = nom; this.description = description; this.prix = prix; } // Getters / Setters // (aucune dépendance Hibernate) }</pre>

Dans la variante `hbm.xml`, la classe `Produit` est un POJO pur (Plain Old Java Object) : elle n'importe rien de `javax.persistence` et ne contient aucune annotation. Toute la configuration du mapping est déportée dans le fichier XML.

3.2 Le fichier `Produit.hbm.xml` — exclusif à la variante `hbm`

Ce fichier, absent de la variante `Annotations`, est le cœur de la variante `hbm.xml`. Il décrit le mapping entre la classe Java et la table MySQL de façon déclarative en XML.

```
<!-- Produit.hbm.xml — fichier de mapping Hibernate classique -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
```

```

"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

<hibernate-mapping package="ma.ensa.models">

  <!-- Équivalent de @Entity @Table(name="produits") -->
  <class name="Produit" table="produits">

    <!-- Équivalent de @Id @GeneratedValue(IDENTITY) @Column("id_produit") -->
    <id name="idProduit" column="id_produit" type="long">
      <generator class="native"/>
    </id>

    <!-- Équivalent de @Column(name="nom", nullable=false, length=150) -->
    <property name="nom" column="nom" type="string" not-null="true" length="150"/>

    <!-- Équivalent de @Column(name="description", length=500) -->
    <property name="description" column="description" type="string" not-null="false" length="500"/>

    <!-- Équivalent de @Column(name="prix", nullable=false) -->
    <property name="prix" column="prix" type="double" not-null="true"/>

  </class>
</hibernate-mapping>

```

Correspondance exacte entre annotations et éléments hbm.xml

Rôle	Annotation JPA (MVC2-Annot.)	Élément hbm.xml (MVC2-hbm)
Déclarer une entité persistante	@Entity	<class name="Produit">
Nom de la table SQL	@Table (name="produits")	table="produits" (attr. de <class>)
Clé primaire	@Id	<id name="idProduit">
Stratégie de génération ID	@GeneratedValue (IDENTITY)	<generator class="native"/>
Nom de la colonne (ID)	@Column (name="id_produit")	column="id_produit" (attr. de <id>)
Propriété simple	@Column (name="nom")	<property name="nom" column="nom"/>
Contrainte NOT NULL	@Column (nullable=false)	not-null="true"
Longueur max.	@Column (length=150)	length="150"
Type de données	(implicite via type Java)	type="string" / type="double"

4. Différence dans hibernate.cfg.xml

Le fichier hibernate.cfg.xml est présent dans les deux variantes avec une configuration de connexion MySQL identique. La seule différence se trouve dans la ligne de déclaration de l'entité mappée, qui change selon l'approche.

MVC2-Annotations — hibernate.cfg.xml	MVC2-hbm.xml — hibernate.cfg.xml
<pre> <hibernate-configuration> <session-factory> <!-- Connexion MySQL (identique) -- > <property name="hibernate.connection .driver_class"> com.mysql.cj.jdbc.Driver </property> <property name="hibernate.connection .url">jdbc:mysql://localhost:3306/ gestion_produits?...</property> <property name="hibernate.dialect"> org.hibernate.dialect.MySQL8Dialect </property> <property name="hibernate.hbm2ddl .auto">update</property> <!-- Référence vers la CLASSE Java --> <mapping class= "ma.ensa.models.Produit"/> </session-factory> </hibernate-configuration> </pre>	<pre> <hibernate-configuration> <session-factory> <!-- Connexion MySQL (identique) -- > <property name="hibernate.connection .driver_class"> com.mysql.cj.jdbc.Driver </property> <property name="hibernate.connection .url">jdbc:mysql://localhost:3306/ gestion_produits?...</property> <property name="hibernate.dialect"> org.hibernate.dialect.MySQL8Dialect </property> <property name="hibernate.hbm2ddl .auto">update</property> <!-- Référence vers le FICHIER XML --> <mapping resource= "ma/ensa/models/Produit.hbm.xml"/> </session-factory> </hibernate-configuration> </pre>

Attribut XML	MVC2-Annotations	MVC2-hbm.xml
Balise	<mapping class="..." />	<mapping resource="..." />
Valeur	Nom qualifié de la classe Java	Chemin classpath vers le fichier XML
Ce que Hibernate lit	Les annotations @Entity, @Table... dans la classe	Le contenu du fichier Produit.hbm.xml

5. Différence dans la configuration des Servlets et Filtres

La deuxième grande différence porte sur la configuration des Servlets et des filtres. Dans la variante Annotations, `@WebServlet` et `@WebFilter` remplacent toute déclaration XML. Dans la variante hbm.xml, on revient au web.xml classique.

5.1 Les Servlets

MVC2-Annotations — @WebServlet	MVC2-hbm.xml — web.xml
<pre>// Annotation sur la classe @WebServlet("/produit") public class ProduitServlet extends HttpServlet { ... } @WebServlet("/login") public class LoginServlet extends HttpServlet { ... } @WebServlet("/logout") public class LogoutServlet extends HttpServlet { ... } // → web.xml ne contient AUCUNE // déclaration de servlet</pre>	<pre><!-- Dans web.xml --> <servlet> <servlet-name>produit</servlet-name> <servlet-class> ma.ensa.web.ProduitServlet </servlet-class> </servlet> <servlet-mapping> <servlet-name>produit</servlet-name> <url-pattern>/produit</url-pattern> </servlet-mapping> <servlet>..LoginServlet..</servlet> <servlet-mapping> <url-pattern>/login</url-pattern> </servlet-mapping> <servlet>..LogoutServlet..</servlet> <servlet-mapping> <url-pattern>/logout</url-pattern> </servlet-mapping></pre>

5.2 Les filtres

MVC2-Annotations — @WebFilter	MVC2-hbm.xml — web.xml
<pre>// Annotation sur la classe @WebFilter("/*") public class AuthFilter implements Filter { ... } @WebFilter("/produit") public class RoleFilter implements Filter { ... } // → Aucune déclaration XML // dans web.xml</pre>	<pre><!-- Dans web.xml --> <filter> <filter-name>AuthFilter</filter-name> <filter-class> ma.ensa.filters.AuthFilter </filter-class> </filter> <filter-mapping> <filter-name>AuthFilter</filter-name> <url-pattern>/*</url-pattern> </filter-mapping> <filter> <filter-name>RoleFilter</filter-name> <filter-class> ma.ensa.filters.RoleFilter </filter-class> </filter> <filter-mapping> <filter-name>RoleFilter</filter-name> <url-pattern>/produit</url-pattern> </filter-mapping></pre>

5.3 Résultat sur le web.xml

MVC2-Annotations — web.xml MINIMAL	MVC2-hbm.xml — web.xml COMPLET
<pre> <!-- Contenu total de web.xml --> <web-app version="3.0"> <display-name> GestionDesProduitsMVC2 </display-name> <welcome-file-list> <welcome-file>index.jsp</welcome- file> </welcome-file-list> <!-- Uniquement les erreurs HTTP --> <error-page> <error-code>404</error-code> <location>/WEB-INF/views/erreur.jsp </location> </error-page> <error-page> <error-code>500</error-code> <location>/WEB-INF/views/erreur.jsp </location> </error-page> </web-app> <!-- ~25 lignes au total --> </pre>	<pre> <!-- web.xml contient tout --> <web-app version="3.0"> <!-- FILTRES --> <filter>AuthFilter</filter> <filter-mapping>/*</filter-mapping> <filter>RoleFilter</filter> <filter-mapping>/produit</filter- mapping> <!-- SERVLETS --> <servlet>LoginServlet</servlet> <servlet-mapping>/login</servlet- mapping> <servlet>LogoutServlet</servlet> <servlet-mapping>/logout</servlet- mapping> <servlet>ProduitServlet</servlet> <servlet-mapping>/produit</servlet- mapping> <!-- ERREURS HTTP --> <error-page>404 → erreur.jsp</error- page> <error-page>500 → erreur.jsp</error- page> </web-app> <!-- ~65 lignes au total --> </pre>

6. Composants identiques dans les deux variantes

Il est important de souligner que la très grande majorité du code est partagée entre les deux variantes. La différence se limite à la couche de mapping et à la configuration.

Composant	Identique ?	Remarque
ProduitDAOImpl.java	✓ Identique	Même code Hibernate (save, get, update, delete, HQL)
HibernateUtil.java	✓ Identique	Même Singleton SessionFactory
ProduitService.java (interface)	✓ Identique	Même contrat CRUD
ProduitServiceImpl.java	✓ Identique	Même délégation au DAO
AuthService.java	✓ Identique	Authentification en mémoire HashMap
Utilisateur.java	✓ Identique	POJO + enum Role
ProduitServlet.java	✓ Identique*	Même logique, seule l'annotation @WebServlet diffère
LoginServlet.java	✓ Identique*	Seule l'annotation @WebServlet diffère
LogoutServlet.java	✓ Identique*	Seule l'annotation @WebServlet diffère
AuthFilter.java	✓ Identique*	Seule l'annotation @WebFilter diffère
RoleFilter.java	✓ Identique*	Seule l'annotation @WebFilter diffère
login.jsp / index.jsp / erreur.jsp	✓ Identiques	Vues inchangées, URLs identiques
pom.xml	✓ Identique	Mêmes dépendances : hibernate-core + mysql-connector

(*) : identique dans la logique, mais sans l'annotation de la variante Annotations.

6.1 ProduitDAOImpl — identique dans les deux variantes

C'est le résultat le plus important à comprendre : même si le mapping est défini différemment (annotations vs hbm.xml), le code du DAO ne change pas du tout. Hibernate résout le mapping en interne, de façon transparente pour le DAO.

```
// ProduitDAOImpl.java – CODE IDENTIQUE dans les deux variantes
public class ProduitDAOImpl implements ProduitDAO {

    private SessionFactory sf = HibernateUtil.getSessionFactory();
```

```
@Override
public void add(Produit p) {
    Transaction tx = null;
    try (Session s = sf.openSession()) {
        tx = s.beginTransaction();
        s.save(p);          // Hibernate génère INSERT INTO produits...
        tx.commit();        // → qu'il connaisse le mapping via @Entity
    } catch (Exception e) { // OU via Produit.hbm.xml
        if (tx != null) tx.rollback();
    }
}

@Override
@SuppressWarnings("unchecked")
public List<Produit> getAll() {
    try (Session s = sf.openSession()) {
        // HQL – identique dans les deux variantes
        return s.createQuery("FROM Produit ORDER BY idProduit").list();
    }
}
// delete, update, getById – identiques aussi
}
```

7. Avantages et inconvénients de chaque approche

Critère	MVC2-Annotations	MVC2-hbm.xml
Simplicité	+ Tout dans le code Java, pas de fichier XML supplémentaire	- Nécessite un fichier hbm.xml par entité
Couplage modèle/Hibernate	- La classe Java dépend de javax.persistence	+ POJO pur, aucune dépendance à Hibernate dans le modèle
Séparation des responsabilités	- Le modèle métier contient des détails de persistance	+ Le mapping est séparé du modèle métier
Flexibilité du schéma	- Doit modifier la classe Java pour changer la table	+ Modifier le hbm.xml sans toucher au code Java
Lisibilité du mapping	+ Visible directement lors de la lecture du code	- Mapping dans un fichier séparé, moins visible
Configuration web.xml	+ Minimal grâce à @WebServlet / @WebFilter	- web.xml plus verbeux (servlets + filtres)
Approche moderne	+ Standard JPA / Jakarta EE actuel	- Approche Hibernate classique (pré-JPA)
Portabilité JPA	+ Compatible avec tout fournisseur JPA (EclipseLink...)	- Spécifique à Hibernate, non portable JPA
Débogage	+ Erreur de mapping localisée dans la classe Java	+ Erreur de mapping localisée dans le fichier XML

8. Flux de traitement — identique dans les deux variantes

Le flux de traitement des requêtes HTTP est absolument identique dans les deux variantes, car la différence ne concerne que le mapping Hibernate, totalement transparent pour les couches supérieures.

- Le navigateur envoie une requête vers /produit?action=...
- AuthFilter intercepte la requête — vérifie la session active
- RoleFilter intercepte si action admin — vérifie le rôle
- ProduitServlet dispatche selon le paramètre action
- La méthode action appelle ProduitService
- ProduitServiceImpl délègue à ProduitDAOImpl
- ProduitDAOImpl ouvre une Session Hibernate et exécute l'opération
- Hibernate génère le SQL en utilisant le mapping (annotations OU hbm.xml)
- La Servlet place les résultats en attributs de requête et forward vers index.jsp
- La vue JSP génère le HTML et le retourne au navigateur

9. Tests fonctionnels

N°	Action	Résultat attendu	Différence entre variantes
T01	Connexion admin/admin123	Redirection → /produit?action=list, rôle ADMIN	Aucune — comportement identique
T02	Connexion user/user123	Mode lecture seule, message bandeau	Aucune
T03	Mauvais identifiants	Message d'erreur, formulaire ré-affiché	Aucune
T04	Ajout produit (ADMIN)	INSERT en BDD MySQL via Hibernate	Aucune — même SQL généré
T05	Modification (ADMIN)	UPDATE en BDD, liste actualisée	Aucune
T06	Suppression (ADMIN)	DELETE en BDD, confirmation JS	Aucune
T07	USER → action=add	Erreur 403 — Accès refusé	Aucune
T08	URL inconnue	Erreur 404 — Page introuvable	Aucune
T09	Recherche par ID existant	Un seul produit affiché (depuis BDD)	Aucune
T10	Déconnexion	Session invalidée, retour /login	Aucune

Tous les tests produisent le même résultat dans les deux variantes, confirmant que le choix du mécanisme de mapping Hibernate est transparent pour l'utilisateur final et pour la logique applicative.

Conclusion

Ce TP a permis de comprendre que le choix du mécanisme de mapping Hibernate est une décision purement technique et architecturale, sans impact sur les fonctionnalités ni sur l'expérience utilisateur.

La variante MVC2-Annotations est l'approche moderne recommandée : elle réduit le nombre de fichiers, garde le mapping visible directement dans le code, et est compatible avec le standard JPA. Elle est aujourd'hui privilégiée dans tous les projets Spring Boot et Jakarta EE.

La variante MVC2-hbm.xml illustre l'approche historique de Hibernate (antérieure à JPA). Elle présente un avantage de séparation des responsabilités — la classe Java reste un POJO pur sans dépendance à Hibernate — mais elle est plus verbeuse et moins portable. Elle reste utile pour comprendre les fondements du framework ou pour travailler sur des projets legacy.

Résumé : les 4 différences clés

- ① Mapping entité : @Entity/@Column dans Produit.java vs Produit.hbm.xml séparé
- ② hibernate.cfg.xml : <mapping class=.../> vs <mapping resource=.../>
- ③ Servlets/Filtres : @WebServlet/@WebFilter vs déclaration XML dans web.xml
- ④ Classe Produit : POJO + annotations JPA vs POJO pur sans aucun import Hibernate

Annexe — Comptes de démonstration

Login	Mot de passe	Rôle	Permissions
admin	admin123	ADMIN	Lister, rechercher, ajouter, modifier, supprimer
user	user123	USER	Lister et rechercher uniquement (lecture seule)